Phase 7: Option Chain (Black–Scholes, Greeks, Virtualization)

What we built

- An **option chain** tab in the middle panel (next to Chart): **calls on the left, strike in the middle, puts on the right**
- **201 strikes** around the current price, for **7 expiries** (the next 4 Fridays, then about 2, 3 and 6 months out)
- Each row is **priced live** with **Black–Scholes** from the Binance spot price: option price, **Delta, Gamma, Theta, Vega**, plus modelled **IV** and **OI**
- **ATM** strike highlighted, **ITM** sides shaded (calls green, puts red), an **ATM** button that scrolls back to it, and a **Γ & Vega** toggle
- **Virtualized with react-window**: only the ~17 visible rows exist in the page, not 201

Files and what each one does

File	Role
`src/trading/blackScholes.ts`	**Pure maths**: `normCdf`, `normPdf`, `priceOption()` → price + Greeks for a call and a put
`src/trading/optionChain.ts`	Expiry dates, strike spacing, the modelled IV smile and OI → 201 `StrikeRow`s
`src/components/OptionChain.tsx`	The table: controls, memoized header, virtualized `List`, one `ChainRow` per visible strike
`src/components/MarketPanel.tsx`	Middle panel: price summary + **Chart / Option chain** tabs
`src/hooks/useNow.ts`	The current time as state, refreshed every minute (for live time decay)

What's real and what's modelled (say this in interviews)

	Source
Spot price	**Real**, live from Binance
Option prices & Greeks	**Real Black–Scholes maths**, on that live spot
Implied volatility	**Modelled**: a typical "smile" around a per-coin base vol
Open interest	**Modelled**: a typical bell shape, with bumps at round strikes
Expiries	Realistic (crypto options expire Fridays 08:00 UTC)

A production chain would take IV/OI (and real bid/ask prices) from an options exchange such as Deribit.

Concepts (theory only)

1. Options basics

- **Call** = the right (not the obligation) to **buy** at the **strike** price, until **expiry**. You profit if the price goes **up** past the strike.
- **Put** = the right to **sell** at the strike. You profit if the price goes **down** below the strike.
- **Premium** = the price you pay for the option (our "Price" column).
- **Moneyness**: | | Call | Put | |–|–|–| | **ITM** (in the money) | strike **<** spot | strike **>** spot | | **ATM** (at the money) | strike ≈ spot | strike ≈ spot | | **OTM** (out of the money) | strike > spot | strike < spot |
- **Intrinsic value** = what you'd get by exercising right now (call: spot – strike, if positive). **Time value** = premium – intrinsic: the value of "what might still happen".
- **Option chain** = all strikes for one expiry in one table: calls | strike | puts.

2. Open Interest (OI)

The number of option contracts **currently open** (bought and not yet closed or expired). It's different from volume (contracts traded today). High OI = lots of interest and liquidity at that strike. It usually piles up **near the money** and at **round numbers** (80,000, 85,000). Our modelled OI copies that shape.

3. Implied Volatility (IV)

Volatility = how much the price tends to move, as an annual % (50% ≈ BTC can easily move ±50% in a year). **Implied** volatility = the volatility you'd have to plug into Black–Scholes to get the option's **market price**. It's the market's expectation of future movement, "implied" by what people pay. - Higher IV → more expensive options (both calls and puts). - **Volatility smile**: in real markets, far-from-the-money strikes have **higher** IV than ATM (crash or moon protection is in demand). Plotted against strike, it curves up at both ends like a smile. - **Skew**: the two ends aren't equal. Downside puts are often pricier (we add a small skew term).

4. Black–Scholes (intuition, no derivation)

A formula for the fair price of a European option from 5 inputs: **spot S, strike K, time T (years), volatility σ, interest rate r**. Assumption: the price follows a random walk (log-normal returns, constant volatility). `Call = S·N(d1) − K·e^(−rT)·N(d2)`. `N()` is the normal-distribution CDF. You can read `N(d2)` as roughly "the probability the call ends ITM". What moves the price: | Input goes up ↑ | Call | Put | |–|–|–| | Spot S | ↑ | ↓ | | Strike K | ↓ | ↑ | | Time T | ↑ | ↑ (usually) | | Volatility σ | ↑ | ↑ | **Sanity check we ran:** S=100, K=100, T=1, σ=20%, r=5% → call **10.4506**, put **5.5735**, the textbook values. And **put–call parity** holds: `C − P = S − K·e^(−rT)`. **Limits:** real volatility isn't constant (hence the smile), prices jump, and it prices *European* options (exercise only at expiry). Crypto options on Deribit are European, so it's a reasonable fit.

5. The Greeks: sensitivities

Greek	Meaning	Typical values	Use
Delta (Δ)	Price change per $1 move in spot	Call 0 → 1, put –1 → 0. **ATM ≈ ±0.5**, deep ITM → ±1	Direction risk. Also a rough probability of expiring ITM. “Delta hedging”
Gamma (Γ)	How fast **delta** changes per $1	Highest **ATM** and near expiry	How unstable your delta is
Theta (Θ)	Price change per **day** passing	Negative for buyers, biggest ATM	**Time decay**: options lose value every day
Vega	Price change per **1 vol point**	Highest ATM and for long expiries	Volatility risk

Gamma and vega are the **same** for a call and a put with the same strike and expiry. In the table, look at how delta crosses ~0.5 at the ATM row, and how theta is largest (most negative) there.

6. Why 201 rows × 15 columns is slow

- ~3,000 DOM cells. Each one costs **memory**, **layout** and **paint**.
- Every price tick recalculates every price → React re-renders every row → the browser re-lays them out. At several ticks per second, that becomes **jank** (Phase 3’s 16.6 ms budget).
- And the user can only **see ~15 rows** at a time anyway.

7. Virtualization (windowing) with react-window

Render **only the rows inside the visible window** (+ a few spare: `overscanCount`). - The list is told the **row count** (201) and the **row height** (24px), so it knows the full scroll height (201 × 24px) without rendering it. - From `scrollTop` it computes which indexes are visible: `first = floor(scrollTop / 24)`. - It renders just those rows, each with an absolute `style.top = index × 24`. As you scroll, the same few row components are **reused** for new indexes. - We checked: **17 of 201 rows** are in the page. The work per tick drops ~12×. **Requirements & pitfalls:** - The list needs a **bounded height**. In a box that grows to fit its content, every row is “visible” and all render. (We hit exactly this on phones and fixed it with a fixed height.) - You **must** apply the `style` prop to each row, or the rows stack on top of each other. - Ctrl+F can’t find off-screen rows, because they don’t exist in the DOM. - Variable row heights are possible, but cost measuring.

8. Memoization: `useMemo` and `React.memo`

- **`useMemo(fn, deps)`** caches a **computed value** until a dependency changes. The strike table (IV smile + OI for 201 strikes) depends on the coin and expiry, **not the live price**, so it’s rebuilt only when you switch expiry, not on every tick.
- **`React.memo(Component)`** skips re-rendering if the props are the same (shallow compare). `ChainHeader` is memoized: the parent re-renders every tick, but the header’s props (`columns`, `template`) are the same objects, so it’s skipped.

- They work **together**: `memo` compares by **reference**, so the `columns` array must be the *same object* across renders. `useMemo` guarantees that. Without it, `filter()` would make a new array each render and memo would never skip.
- react-window **already** wraps each row in `memo` and compares `rowProps` value by value.
- **Don't memoize everything:** memo has its own cost (storing and comparing). Use it where a re-render is expensive or very frequent.

9. Smaller ideas in this phase

- **Compute only what's visible:** Black–Scholes runs **inside the row**, so only visible rows are priced (~20 per tick, not 201).
- **Stable anchor:** strikes are built around the spot price **when the chain opened** (`useState(spot)` uses its argument only on the first render), so rows don't shift as the price moves.
- **Pure render:** calling `Date.now()` during render is impure (oxlint flagged it). The time lives in state (`useNow`), and a timer updates it.
- **Columns as data:** each column is `{label, call(), put()}`. Header and rows loop over the same list, and puts use it **reversed** so "Price" sits next to the strike on both sides.
- **Conditional rendering unmounts:** switching tabs removes the chart (its socket closes in the cleanup) or the chain.

Try it yourself

1. Market panel → **Option chain**. Find the yellow **ATM** strike. Strikes go up as you scroll down, so calls are ITM in the rows **above** ATM (strike < spot, green shade) and puts are ITM in the rows **below** it (strike > spot, red shade).
2. Watch **Delta**: ~0.5 at ATM, → 1.0 for deep ITM calls, → 0 for far OTM. Puts mirror it from –1 to 0.
3. Compare **expiries**: the nearest expiry has smaller prices, bigger (more negative) theta near ATM and higher gamma. 6 months out has big prices and big vega.
4. Tick **Γ & Vega** to add the two columns. Scroll far away, then press **ATM**.
5. Look at **IV** across strikes: lowest near ATM and rising both ways (the smile).
6. DevTools → Elements: scroll the chain and watch the rows get **reused**. There are only ~17 row `div`s. React DevTools → "Highlight updates" on a tick: only the visible rows flash, and the header doesn't.

Resume mapping

> "**option chain** with **OI**, **IV** and **Black–Scholes Greeks**" → `blackScholes.ts`, `optionChain.ts`, `OptionChain.tsx` "**react-window virtualization** and **memoization** for 200+ strikes" → `List` + `useMemo` + `memo(ChainHeader)` Honest framing: "Greeks computed client-side with Black–Scholes from the live spot price. IV and OI are modelled."

Interview Qs

1. **Call vs put, ITM vs OTM?** A call is the right to buy, a put the right to sell, at the strike. A call is ITM when strike < spot, a put when strike > spot.
2. **What is implied volatility?** The volatility that makes Black–Scholes match the market price. It's the market's forecast of movement. Higher IV means pricier options.
3. **What's the volatility smile?** IV is higher for far OTM/ITM strikes than ATM, because markets price tail risk above what Black–Scholes' constant-vol assumption gives.
4. **Explain the Greeks.** Delta: sensitivity to spot. Gamma: sensitivity of delta. Theta: decay per day. Vega: sensitivity to 1 vol point. Which is biggest at ATM? Gamma, theta and vega.
5. **Why is ATM delta about 0.5?** It's roughly a 50/50 chance of finishing in the money.
6. **How did you verify your Black–Scholes code?** Against textbook values (10.4506 / 5.5735) and put–call parity.
7. **How do you render 200+ strikes smoothly with live prices?** Virtualization renders only the visible ~17 rows. Pricing happens inside the visible rows only. The static strike data is in `useMemo`. The header is `memo`'d.
8. **How does virtualization work?** A fixed row height gives the total scroll height. From `scrollTop` you compute the visible index range, render only those rows absolutely positioned, and reuse them as you scroll.
9. **When doesn't `React.memo` help?** When props are new objects/functions on every render (fix with `useMemo` / `useCallback`), or when the component re-renders from its own state or context.
10. **Downsides of virtualization?** Ctrl+F and accessibility tools can't see off-screen rows, it needs a bounded container, variable heights are harder, and you can see blank flashes on very fast scrolls (that's what overscan is for).